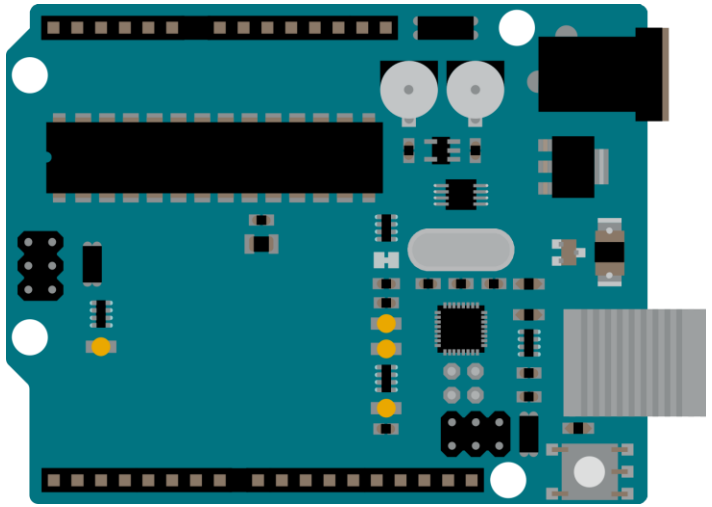
Microcontroller

Mini Project 1
Dot Matrix Display



Session 5

ATmega328P with bare-metal AVR C

The project and its evidence

One letter and three distinct animations

A pin map, current budget and test record

Prerequisite check

DDRx chooses direction

PORTx writes an output level

F_CPU describes the clock used by delay code

One pixel is a current path

A column supplies current to an LED anode

A selected row gives its cathode a path to ground

The 8 by 8 connections

	C0	C1	C2	C3	C4	C5	C6	C7
R0	0	0	0	0	0	0	0	0
R1	0	0	0	0	0	0	0	0
R2	0	0	0	0	0	0	0	0
R3	0	0	0	0	0	**1**	0	0
R4	0	0	0	0	0	0	0	0
R5	0	0	0	0	0	0	0	0
R6	0	0	0	0	0	0	0	0
R7	0	0	0	0	0	0	0	0

Row 0 at top. Bit 0 at left.

Eight common anode columns

Eight common cathode rows

Pixel coordinates are row then column

Control levels and LED state

Row control	Column output	Pixel
LOW	LOW	Off
LOW	HIGH	Off
HIGH	LOW	Off
HIGH	HIGH	On

Demonstration 1 One pixel

	C0	C1	C2	C3	C4	C5	C6	C7
R0	0	0	0	0	0	0	0	0
R1	0	0	0	0	0	0	0	0
R2	0	0	0	0	0	0	0	0
R3	0	0	0	0	0	**1**	0	0
R4	0	0	0	0	0	0	0	0
R5	0	0	0	0	0	0	0	0
R6	0	0	0	0	0	0	0	0
R7	0	0	0	0	0	0	0	0

Row 0 at top. Bit 0 at left.

DEMO_STAGE = 1

R3 active and column byte 0x20

Only pixel R3,C5 lights

Demonstration 2 One row

	C0	C1	C2	C3	C4	C5	C6	C7
R0	0	0	0	0	0	0	0	0
R1	0	0	0	0	0	0	0	0
R2	0	0	0	0	0	0	0	0
R3	0	**1**	0	**1**	0	**1**	0	**1**
R4	0	0	0	0	0	0	0	0
R5	0	0	0	0	0	0	0	0
R6	0	0	0	0	0	0	0	0
R7	0	0	0	0	0	0	0	0

Row 0 at top. Bit 0 at left.

DEMO_STAGE = 2

0xAA selects C1,C3,C5,C7

R3 remains the only active row

Column wiring

Column	AVR pin	Uno label	Series resistor
C0	PD2	D2	330 ohm
C1	PD3	D3	330 ohm
C2	PD4	D4	330 ohm
C3	PD5	D5	330 ohm
C4	PD6	D6	330 ohm
C5	PD7	D7	330 ohm
C6	PB0	D8	330 ohm
C7	PB1	D9	330 ohm

Row wiring

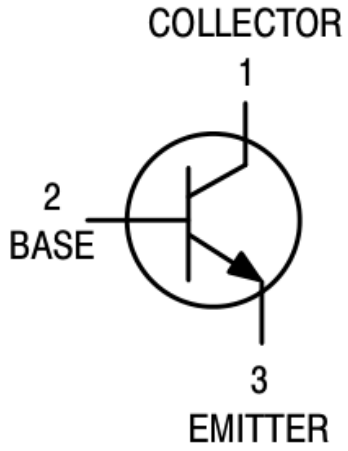
Row	AVR pin	Uno label	Control connection
R0	PB2	D10	1k to base
R1	PB3	D11	1k to base
R2	PB4	D12	1k to base
R3	PB5	D13	1k to base
R4	PC0	A0	1k to base
R5	PC1	A1	1k to base
R6	PC2	A2	1k to base
R7	PC3	A3	1k to base

The NPN row driver

Collector connects to row cathodes

Emitter connects to common ground

Base receives control through 1k



A current estimate before a full row

5 V supply, red LED about 2.0 V

Assume about 0.2 V across the active NPN

$I_{LED} \approx (5 - 2.0 - 0.2) / 330 \approx 8\ mA$

The current budget

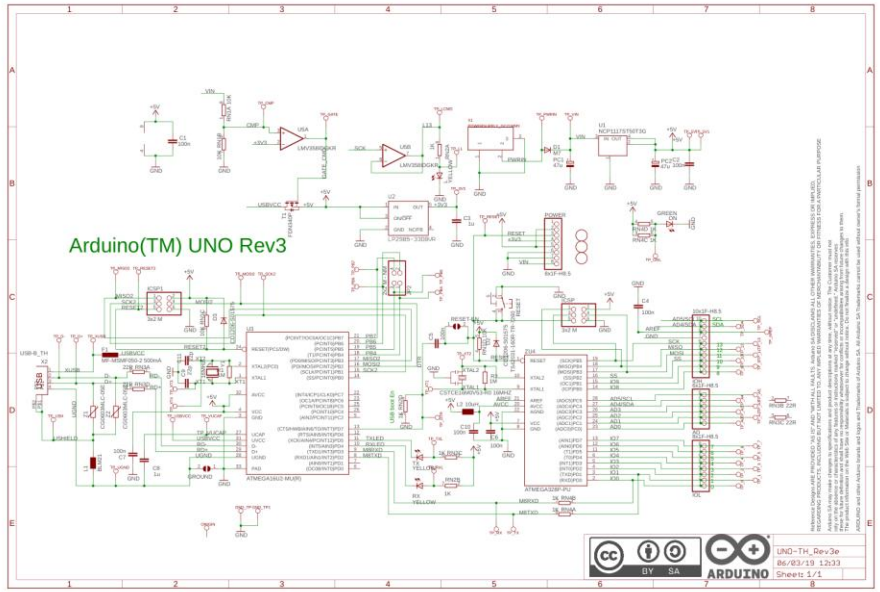
Path	Nominal during row slot
One column GPIO	About 8 mA
Eight LEDs in selected row	About 64 mA through NPN
Selected base GPIO	About 4.3 mA
One LED average at 1/8 duty	About 1 mA

Power reset and clock

The Uno already provides these support circuits

Its ATmega328P runs at 16 MHz

A later custom board needs its own support circuit



Support circuit detail is in the setup guide.

The wiring check before power

All grounds share a reference

Every column has its resistor

No row bypasses its NPN driver

Collector base emitter labels match the part

Row scanning

Slot	Selected row	Column byte
0	R0	0xFF
1	R1	0xFF
2	R2	0xFF
3	R3	0xFF
4	R4	0xFF
5	R5	0xFF
6	R6	0xFF
7	R7	0xFF

Demonstration 3 Slow rows

DEMO_STAGE = 3

Each row stays selected for 250 ms

Eight rows take about 2 seconds

Blanking between rows

Step	Rows	Columns
1. Rows off	All off	Old byte
2. 5 us blank	All off	Old byte
3. Write byte	All off	Next row byte
4. Select and dwell	One row on	Next row byte

Full-display refresh

$T_{scan} \approx 8 \times T_{row}$

250 ms per row gives about 0.5 Hz

1 ms per row gives about 125 Hz

Refresh rate and animation rate

Quantity	What changes	Example
Row selection rate	Active row	About 1000 slots/s
Display refresh rate	Complete scan	About 125 scans/s
Animation image rate	Bitmap content	About 5 images/s

A still image needs repeated scanning

The bitmap can stay unchanged

The row scan continues for the entire hold time

Timing checkpoint

2 ms row dwell and eight rows

100 complete scans of one image

Calculate refresh rate and image hold time

One row is one byte

	C0	C1	C2	C3	C4	C5	C6	C7
R0	0	0	0	**1**	**1**	0	0	0
R1	0	0	**1**	0	0	**1**	0	0
R2	0	**1**	0	0	0	0	**1**	0
R3	0	**1**	0	0	0	0	**1**	0
R4	0	**1**	**1**	**1**	**1**	**1**	**1**	0
R5	0	**1**	0	0	0	0	**1**	0
R6	0	**1**	0	0	0	0	**1**	0
R7	0	0	0	0	0	0	0	0

Row 0 at top. Bit 0 at left.

Eight bits select eight columns

Eight row bytes describe one 8 by 8 image

```
R0   0x18
R1   0x24
R2   0x42
R3   0x42
R4   0x7E
R5   0x42
R6   0x42
R7   0x00
```

The bit-order convention

Column	C0	C1	C2	C3	C4	C5	C6	C7
Bit	0	1	2	3	4	5	6	7
Weight	1	2	4	8	16	32	64	128

C0 is left and uses bit 0

C7 is right and uses bit 7

Normal binary notation writes bit 7 first

Encoding a row

C1,C3,C5,C7 should light

2 + 8 + 32 + 128 = 170

170 decimal = 0xAA

Arrays and function arguments

uint8_t image[8] stores eight row bytes

image[row] selects one byte

A function can receive the address of the image

```
const uint8_t image[8] = {
  0x18,0x24,0x42,0x42,
  0x7E,0x42,0x42,0x00
};
scan_image(image, 0u);
```

A byte spans two GPIO ports

Bits 0 to 5 occupy PD2 to PD7

Bits 6 and 7 occupy PB0 and PB1

Masks preserve the other port bits

```
PORTD = (PORTD & 0x03)
      | ((bits << 2) & 0xFC);
PORTB = (PORTB & 0xFC)
      | ((bits >> 6) & 0x03);
```

Safe initialization

Output latches begin with rows and columns off

DDRx then enables the required outputs

```
rows_off();
columns_write(0u);
DDRD |= 0xFCu;
DDRB |= 0x3Fu;
DDRC |= 0x0Fu;
```

The row-slot function

Blank first

Write one column byte

Select one row and wait

Turn that row off

```
rows_off();
_delay_us(5);
columns_write(bits);
row_on(row);
_delay_ms(1);
rows_off();
```

The complete scan

Each row reads its own bitmap byte

The loop visits rows 0 through 7

```
for (uint8_t row=0; row<8; ++row) {
    row_slot(row, image[row], slow);
}
```

Six stages on one circuit

Stage	Program	Expected result
1	matrix_pixel	R3,C5
2	matrix_row	R3 alternate pixels
3	matrix_slow_rows	Solid rows travel down
4	matrix_slow_bitmap	A appears row by row
5	matrix_static	Stable letter A
6	matrix_animation	Three repeated animations

Animation 1 Beating heart

Small heart

	C0	C1	C2	C3	C4	C5	C6	C7
R0	0	0	0	0	0	0	0	0
R1	0	0	0	0	0	0	0	0
R2	0	0	1	0	1	0	0	0
R3	0	1	1	1	1	1	0	0
R4	0	1	1	1	1	1	0	0
R5	0	0	1	1	1	0	0	0
R6	0	0	0	1	0	0	0	0
R7	0	0	0	0	0	0	0	0

Large heart

	C0	C1	C2	C3	C4	C5	C6	C7
R0	0	0	0	0	0	0	0	0
R1	0	1	1	0	0	1	1	0
R2	1	1	1	1	1	1	1	1
R3	1	1	1	1	1	1	1	1
R4	0	1	1	1	1	1	1	0
R5	0	0	1	1	1	1	0	0
R6	0	0	0	1	1	0	0	0
R7	0	0	0	0	0	0	0	0

Animations 2 and 3

A vertical bar moves right then left

A box outline grows then shrinks

Each sequence changes visible content over time

Debugging starts with the symptom

Symptom	First targeted check
Mirrored F	C0/C7 convention and wiring
One row missing	Collector/base/emitter and row mask
Ghost pixels	Rows off during column changes
Dark during a delay	Continuous scan service

Expected timing and measurements

R0 repeats once per complete scan

Nominal scan period is about 8 ms

AVR simulation gives about 8.085 ms per scan

UART before the laboratory

Serial means bits travel one after another

Asynchronous UART has no shared clock wire

Both ends agree on baud rate and frame format

TX RX and common ground

ATmega328P signal	Uno label	External peer connection
PD1 / TXD	D1	Peer RX
PD0 / RXD	D0	Peer TX
GND	GND	Peer GND

An 8N1 frame for letter A

Start	D0	D1	D2	D3	D4	D5	D6	D7	Stop
0	1	0	0	0	0	0	1	0	1

8 data bits, no parity, 1 stop bit

Start LOW, data least-significant bit first, stop HIGH

Letter A is byte 0x41

Baud rate and UBRR

At 9600 baud one bit takes about 104 us

Normal mode uses a 16× timing factor

16 MHz gives UBRR = 103 for about 9600 baud

UBRR ≈ F_CPU / (16 × baud) − 1

ATmega328P USART registers

Register	Data address	Purpose
UCSR0A	0xC0	Status flags and speed
UCSR0B	0xC1	Enables
UCSR0C	0xC2	Frame format
UBRR0L	0xC4	Divider low byte
UBRR0H	0xC5	Divider high byte
UDR0	0xC6	Send or receive byte

9600 baud and 8N1 initialization

U2X0 = 0 selects normal speed

UCSZ01 and UCSZ00 = 1 select 8 bits

RXEN0 and TXEN0 enable the hardware

```
UCSR0A = 0;
UBRR0H = 0;
UBRR0L = 103;
UCSR0C = (1<<UCSZ01)|(1<<UCSZ00);
UCSR0B = (1<<RXEN0)|(1<<TXEN0);
```

Polling transmit

UDRE0 means the transmit data buffer has space

Writing UDR0 queues one byte

The peripheral sends its 8N1 frame

```
while ((UCSR0A & (1u<<UDRE0)) == 0) { }
UDR0 = byte;
```

Polling receive and the LED command

RXC0 means a received byte is available

Reading UDR0 consumes that byte

Letter a turns PB5 on; other commands turn it off

```
while ((UCSR0A & (1<<RXC0)) == 0) { }
uint8_t byte = UDR0;
if (byte == 'a') { PORTB |= (1<<PB5); }
```

Polling echo

Receive one byte

Wait for transmit buffer space

Send the same byte back

```
uint8_t errors;
uint8_t byte = uart_rx(&errors);
if (errors == 0) { uart_tx(byte); }
```

Receive interrupts preview

RXCIE0 enables the receive interrupt source

sei() enables global interrupts

A short ISR captures received data for main()

```
UCSR0B |= (1<<RXCIE0);
sei();

ISR(USART_RX_vect) {
   /* Read UDR0 and save the byte promptly. */
}
```

The Lab 5 readiness check

9600 baud, 8N1, correct port

Transmit CE-LAB, LED command, polling echo

Receive-interrupt echo uses the companion code

UART-only Uno has the matrix harness disconnected

What students should remember

One active row and one column byte select pixels

A static bitmap still needs continuous refresh

Animation changes content at a separate rate

Current limiting and row drivers need a calculation

UART uses agreed framing, baud and ready flags

Why resistor location matters

One resistor per column works with one active row

Two active rows would share a column resistor

One shared row resistor makes brightness depend on lit-pixel count

A conservative current bound

Supply at most 5.25 V

Red LED forward voltage at least 1.6 V

330 ohm resistor at −5% gives 313 ohm

Ignoring NPN drop gives $I_{LED} \leq 11.67$ mA

Transistor drive needs a check

Nominal row current about 40 mA

Nominal base current about 4.3 mA

Current ratio is about 9.3

Measure collector voltage with all eight pixels on

A wrong F_CPU creates two failures

F_CPU guides delay and baud calculations

The clock hardware sets real elapsed time

16 MHz code on an 8 MHz CPU roughly doubles delays

A blocking animation mistake

One scan finishes with all rows off

A 200 ms delay stops refreshing

The stored image survives but the LEDs go dark

```
scan_image(image, 0u);
_delay_ms(200);  /* explained failure */

/* Repair: hold_image(image, 25u); */
```

Bitmap memory on a small AVR

One image uses 8 bytes

Four outlines use 32 bytes

Ordinary const does not automatically mean Flash-only

```
#include <avr/pgmspace.h>
const uint8_t glyph[8] PROGMEM = { /* bytes */ };
uint8_t bits = pgm_read_byte(&glyph[row]);
```

Changing a frame at a scan boundary

A complete scan should use one consistent bitmap

Change image selection between complete scans

Double buffering helps when another task edits pixels

Four LEDs as a hardware fallback

Build R0,R1 with C0,C1

Keep the same pin conventions and NPN topology

Use the full offline grid for 8-byte representation

UART bytes and terminal line endings

Character	Hex byte	Companion LED action
a	0x61	On
b	0x62	Off
A	0x41	Off
CR	0x0D	Ignore
LF	0x0A	Ignore

UDRE0 and TXC0 answer different questions

Item	Meaning
UDRE0	Another byte can enter the transmit buffer
TXC0	Transmission has completed
Write UDR0	Feed transmitter
Read UDR0	Consume received byte

The receive-interrupt companion

The ISR checks status and reads UDR0

A small queue saves bytes for main()

Main waits for UDRE0 before transmitting

A finite queue can still overflow

Discussion questions

Why can a camera disagree with our eyes?

Why does a still image need CPU work?

When would a timer help the design?